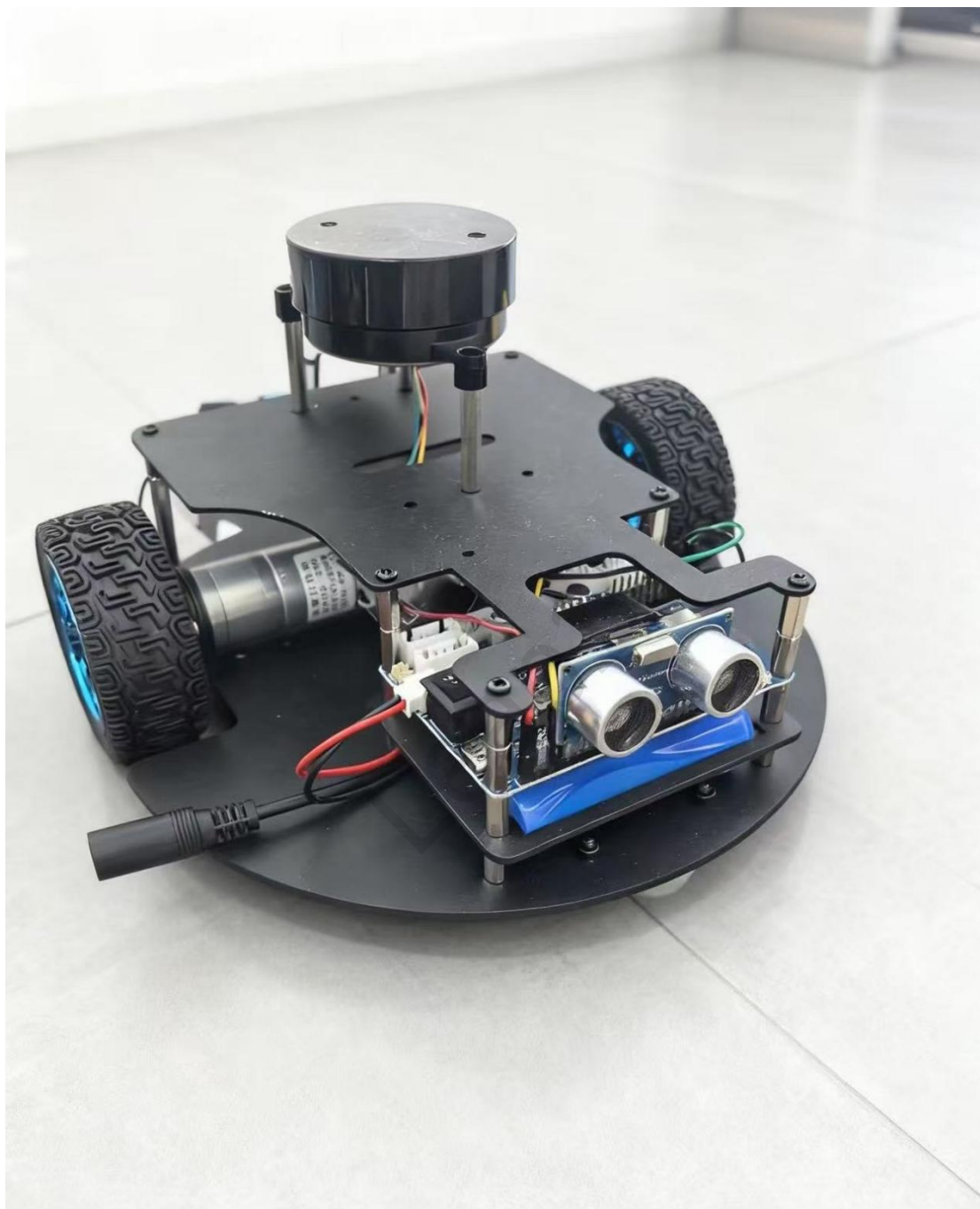


Leap1 开源版产品手册



目录

一、Leap1 基本信息	4
1.1 Leap1 介绍	4
1.2 Leap1 的用途	4
1.3 Leap1 硬件规格	4
二、Leap1 使用介绍	11
2.1 固件烧录和配置的方法	12
三、快速上手指南	16
3.1 查询虚拟机的 IP	18
3.2 连接 leap1 的配置工具	18
3.3 连接 microros 代理	19
3.4 启动激光雷达驱动	20
3.5 启动 <code>odo->base_footprint</code> 的坐标变换	21
3.6 开启 gmapping 建图:	21
3.7 开启 nav2 导航	22
四、Leap1 组件测试	23
4.1 Microros 通讯测试	23
4.2 Gmapping 建图	26
4.3 Nav2 导航测试	33

欢迎查阅 MATUO LEAP1 使用手册,为了让您更好地使用本产品,请在第一次操作之前,认真阅读本手册,并严格按照操作规范使用本产品。

您的 LEAP1 机器人的设计和配置可能与本手册中的不同,具体的操作细节有所区别,不必完全参照手册进行操作。

警告

注意锂电池的充电需要有人在一旁,以防任何意外造成的损失
小心接线和供电,以防短路或其他原因造成损坏

Leap1

一、Leap1 基本信息

1.1 Leap1 介绍

Leap1 采用了 ESP32 作为主控，集成 Xtensa® 32 位双核处理器，内嵌 448KB ROM、520KB SRAM，内置 Wi-Fi 802.11 b/g/n 和蓝牙 4.2/5.0 无线连接模块。具有丰富的外设控制能力。激光雷达可选配三角测距的 YDLIDAR-X2 或者 TOF 的 LD06。无需搭载计算平台，采用虚拟机作为主控

1.2 Leap1 的用途

我们可以用它实现：
ROS2 学习入门机器人
路径规划算法验证
地图构建算法验证等

1.3 Leap1 硬件规格

硬件规格参数	
主控芯片	ESP32-WROOM-32E-H4
CPU	RISC-V 32 位双核处理器
内存	320KB SRAM 128KB ROM
WIFI+蓝牙	Wi-Fi 2.4G,Bluetooth 5.4
激光雷达	YDLIDAR-X2
减速电机	370，减速比 1： 46，额定转速 130RPM，额定电压 12V
显示屏	0.96 寸 OLED
电池组	12V 可充电锂电池

1.3.1 直流编码减速电机

Leap1 采用的驱动核心为 JGA25-370 直流减速电机。其额定工作电压为 **12V DC**，在此电压下可输出 **130 RPM** 的额定转速。该电机集成了行星齿轮减速箱，减速比为 **1: 46**。具备结构紧凑、输出扭矩大、运行平稳可靠等特点。



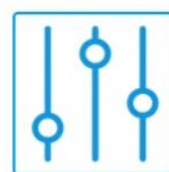
刹车功能



内置有感驱动



接线图



调速功能



FG 信号反馈



过流保护



过压保护



过温保护

Leap1 采用的减速电机具有如下特性

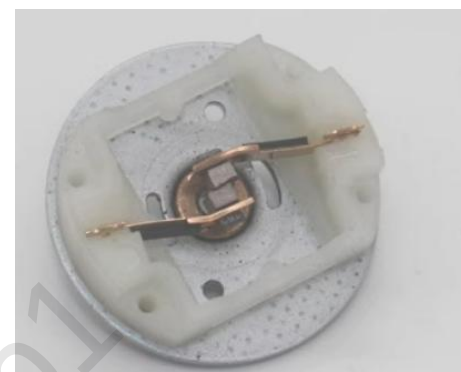
1. 纯铜线圈

采用高精度一次成型转子优质材料打造，导电系数高，耐磨高寿命，同时采用纯铜组，电组小，发热量低，损耗低，小体积、运行平稳，很有劲。



2. 优质碳刷

在其他材料达到要求的前提下，具有较好的换向性能使用寿命长，导电导热以及润滑性能良好。碳刷的耐磨程度直接决定了电机能使用多长时间。



3. 高强度磁瓦

寿命长，动力强劲，力矩更大采用全封闭结构，防止灰尘进入电机内部，结构简单、维修方便体积小、可有效地减轻重量，节省空间，使用可靠。



4. 全金属齿轮

减速箱全部采用优质金属齿轮打造，具有抗冲击性好，经久耐用降低噪音，从而延迟减速箱的使用寿命。



1.3.2 YDLIDAR-X2 激光雷达

Leap1 采用精选的库存新 YDLIDAR-X2 激光雷达，该雷达基于三角测距原理，实现高频高精度的距离测量，在测距的同时，机械结构 360 度旋转不断获取角度信息，从而实现了 360 度扫描测距，输出扫描环境的点云数据。



规格参数

性能参数如下：

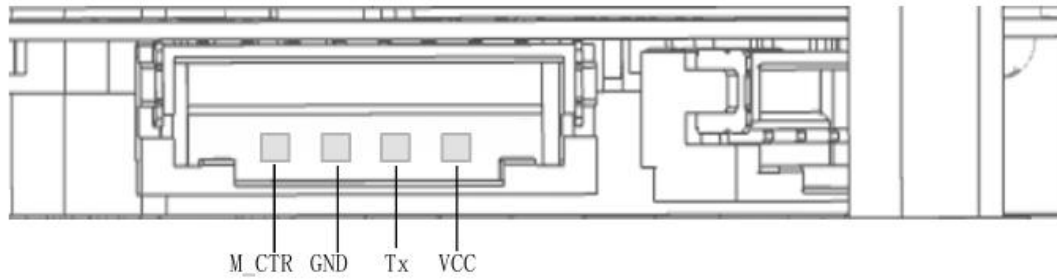
项目	最小值	典型值	最大值	单位	备注
测距频率	/	3000	/	Hz	每秒测距 3000 次
扫描频率	5	6	8	Hz	需要接入 PWM 信号，推荐使用转速 6Hz
测距范围	0.12	/	8	m	室内环境，80%反射率物体
扫描角度	/	0-360	/	°	/
绝对误差	/	2	/	cm	测距小于等于 1m
相对误差	/	3.5%	/	/	1m<测距<=6m
俯仰角	0.25	1	1.75	°	/
角度分辨率	0.6(5Hz)	0.72(6Hz)	0.96(8Hz)	°	不同扫描频率

电气参数

项目	最小值	典型值	最大值	单位	备注
供电电压	4.8	5	5.2	V	过高会损坏设备，过低会影响性能
供电电流	1000	/	/	ma	给雷达供电的电源需要有足够的驱动能力
工作电流	/	300	500	ma	设备正常工作的电流

接口定义

X2 对外提供了 MX1.25-4P 母座接口，该接口有系统供电、数据通信和电机控制的功能接口。串口波特率为 115200



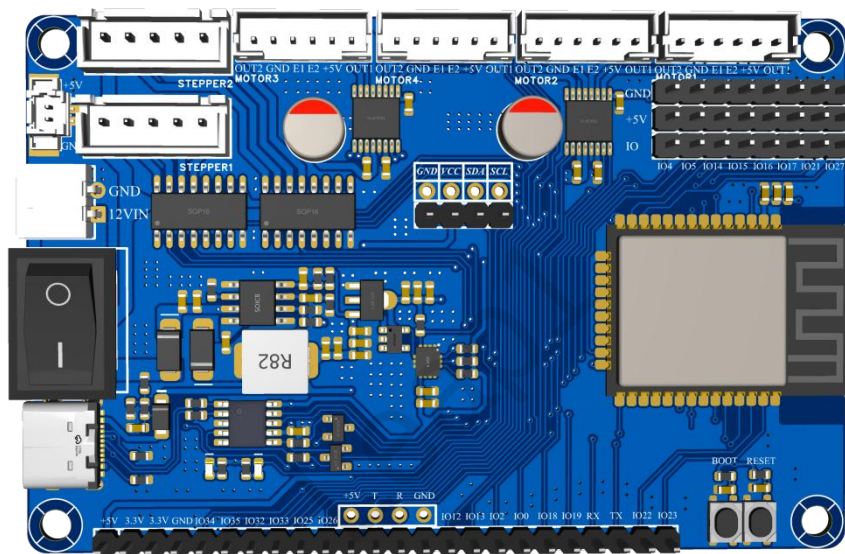
管脚	类型	描述	默认值	范围	备注
VCC	供电	供电+	5V	4.8V-5.2V	/
TX	输出	串口输出	/	/	雷达到外设
GND	供电	供电-	0v	0v	/
M_CTRL	输入	电机转速控制	1.8V	0V-3.3V	PWM 或者电压调速

Leap1 中 M_CTRL 引脚默认接在 3.3V，即给最大转速，若过快可将 M_CTRL 引脚悬空，悬空则为 1.8V

1.3.3 Leap1 microRos 控制板

Leap-1 ROS 控制板，既是轻量级的 ROS2 驱动控制器，也是 ESP32 双核开发板。它的板载部分集成了 ESP32 核心模组控制单元，还配备了电机驱动、舵机驱动、六轴 IMU 姿态传感器等关键外设;通讯方面，支持 WiFi、蓝牙、串口等功能;驱动能力上，最大可驱动四路编码器电机、一路激光雷达，也能适配 ESP32-WiFi 摄像头图传模块;供电接口采用 XH2.54 规格。可支持多种类型包括：四轮差速/麦克纳姆轮/两轮差速/阿克曼等结构。

板载资源说明



电池供电接口：连接 12V 锂电池（XH2.54 接口）为机器人供电。

电源开关：机器人电源总开关。

复位按键：ESP32S3 控制芯片的复位按钮。

雷达接口：连接 YDLIDAR-X2 激光雷达。

Type-C 串口：用于烧录固件、配置参数、串口通讯等功能。

电源指示灯和 MCU 指示灯：指示 Leap1 当前状态。

BOOT 按键：ESP32S3 控制芯片的 BOOT 按键，也可以用到自定义按键用。

电机接口：连接 370 编码器电机。

ESP32 控制芯片：microROS 控制板的主要控制芯片，负责管理板载上的所有外设功能。

六轴 IMU 芯片：提供机器人当前的姿态信息。

PWM 舵机接口：用于连接 PWM 舵机

超声波接口：用于连接超声波

注：丝印 SCL 和 SDA 因生产原因贴反，实际 SDA 和 SCL 是相反的，PWM 接口具体查看原理图开放了哪些 IO，仅二驱可以使用 PWM 舵机，四驱无法使用 PWM 舵机。

板载的 ESP32 通过 microros 与虚拟机或其他可运行 ROS2 的设备通讯，Leap1 ROS 控制板出厂固件默认为二驱控制板（固件请从客服领取，默认不刷固件）。ROS 版本为 humble。

Leap1

二、Leap1 使用介绍

环境配置是 Leap1 正常运行的前提，核心在于确保运行载体的系统版本与 ROS2 版本满足硬性要求，避免因版本不兼容导致设备无法启动、功能异常等问题。

请务必确认您所使用的虚拟机或实体机，已成功安装 **Ubuntu 22.04 操作系统**。其他 Ubuntu 版本（如 20.04、24.04）或非 Ubuntu 类 Linux 系统，可能存在驱动适配不全、依赖包安装失败等风险，不建议使用。在系统满足要求的基础上，需进一步安装 **ROS2 Humble**。

获取 Leap1 的固件和相关资料

资料主要包含

ROS 驱动板源码

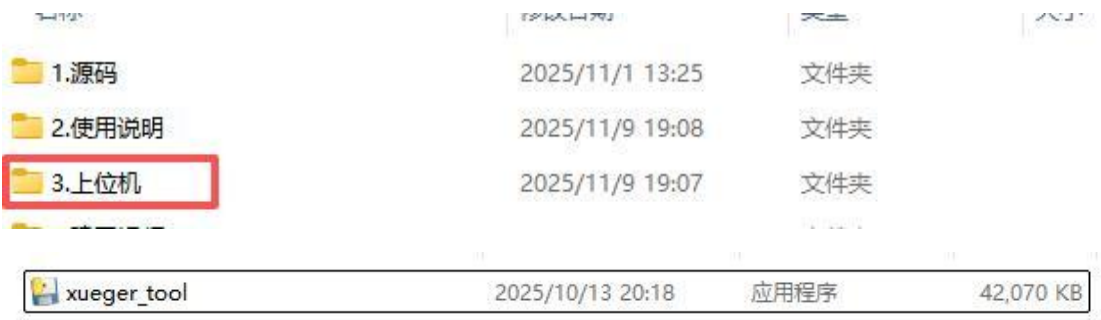
ROS2 功能包源码

官方工具：主要包括 Leap1 使用过程中需要用到的软件

资料请找客服下载领取

Leap1

2.1 固件烧录和配置的方法

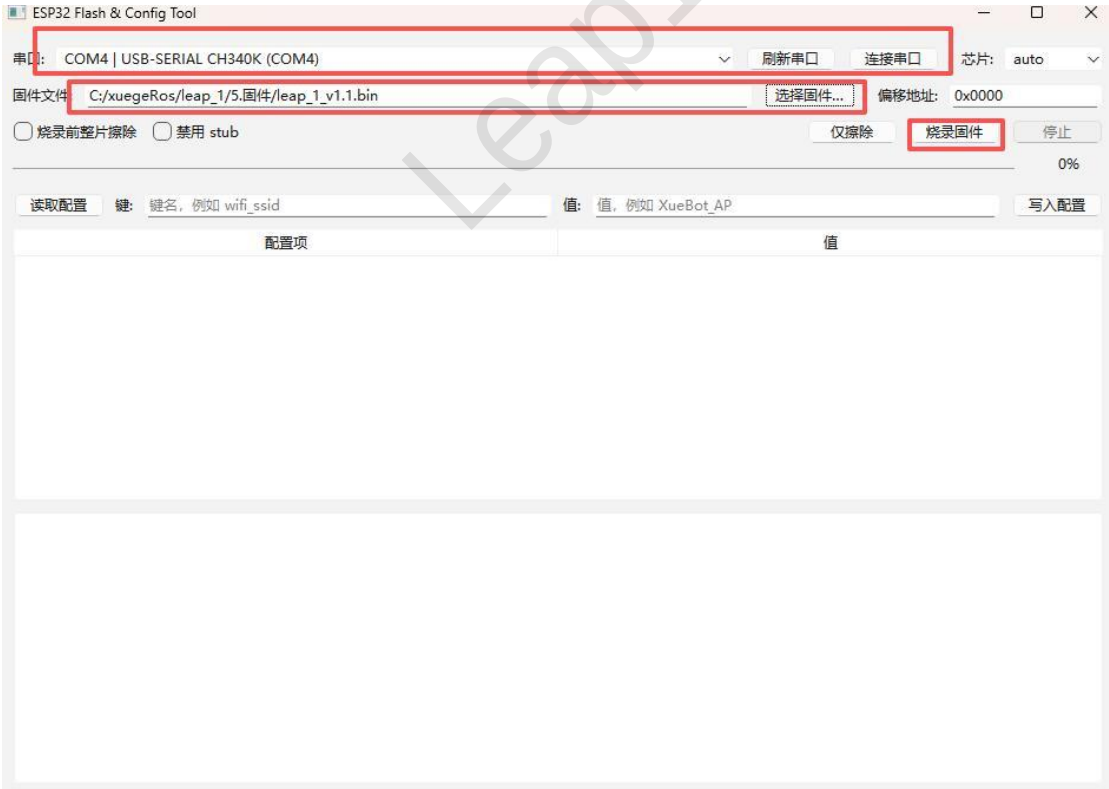


2.1.1 固件烧录步骤

为确保设备正常运行，请在开始烧录前准备好数据线、电脑端烧录工具及正确的固件文件。

1. 连接串口设备

使用 USB 数据线将设备连接到电脑。本设备采用 CH340K 串口芯片，电脑识别后会自动分配串口号（如 COM3、COM5 等），串口号以电脑显示为准。若电脑无法识别设备，请先安装 CH340 驱动程序。



2. 打开烧录工具

在主界面中找到“连接串口”按钮，点击后选择上一步中确认的串口号。当连接成功后，软件会显示“串口连接成功”或类似提示。

3. 选择固件文件

点击“选择固件”按钮，打开文件浏览窗口。在资料包中依次进入：5.固件 文件夹，找到对应型号的固件文件。确认文件路径正确后，点击“打开”加载固件。

4. 开始烧录

确认串口已连接、固件文件已选择完毕后，点击“烧录固件”按钮。软件将自动进行擦除、写入和校验操作。整个过程大约持续数十秒，请勿在烧录过程中断开设备。烧录完毕后会显示如下图所示状。

2.1.2 参数配置步骤

固件烧录完成后，需要进行参数配置，以确保设备能够正确连接 Wi-Fi 网络并与上位机通信。

1. 打开参数配置界面

保持设备与电脑连接，在烧录工具主界面点击“读取参数”按钮。软件会自动从设备中读取当前配置，并显示在参数配置表中。请耐心等待数秒，直到表格内容加载完毕。

串口: COM4 | USB-SERIAL CH340K (COM4) 刷新串口 断开串口 芯片: auto

固件文件: C:/xuegeRos/leap_1/5.固件/leap_1_v1.1.bin 选择固件... 偏移地址: 0x0000

☐ 烧录前整片擦除 ☐ 禁用 stub 仅擦除 烧录固件 停止

0%

读取配置 键: 键名, 例如 wifi_ssid 值: 值, 例如 XueBot_AP 写入配置

配置项	值
first_startup	0
serial_baud	921600
serial_id	0
wifi_ssid	xuegeros
wifi_pswd	88888888
wifi_ap_ssid	XUEBOT_82B4
wifi_ap_pswd	

\$odom_c_frameid=base_footprint
\$twist_topic=cmd_vel
\$odom_pub_period=50
\$pid_kp=0.62
\$pid_ki=0.13
\$pid_kd=0.00[150888][V][WiFiGeneric.cpp:363] _arduino_event_cb(): STA Disconnected: SSID: xuegeros, BSSID: 00:00:00:00:00:00, Reason: 201
[150899][D][WiFiGeneric.cpp:1040] _eventCallback(): Arduino Event: 5 - STA_DISCONNECTED
[150908][W][WiFiGeneric.cpp:1062] _eventCallback(): Reason: 201 - NO_AP_FOUND
[150915][D][WiFiGeneric.cpp:1086] _eventCallback(): WiFi AutoReconnect Running
[150924][V][WiFiGeneric.cpp:97] set_esp_interface_ip(): Configuring Station static IP: 0.0.0.0, MASK: 0.0.0.0, GW: 0.0.0.0
WIFI EVENT!
[150942]@202>D:WIFI DISCONNECTED REASON:201

2. 参数配置项说明

参数名称	说明	示例
wifi_ssid	设备要连接的 Wi-Fi 名称（热点名称）	xuegeros
wifi_pswd	对应的 Wi-Fi 密码	88888888
udpsvr_ip	上位机（虚拟机）所在的 IP 地址	192.168.3.105
udpsvr_port	设备与上位机通信使用的 UDP 端口号，默认无需修改	8888

注意：Wi-Fi 名称与密码区分大小写，请准确填写；若虚拟机 IP 地址填写错误，将导致通信失败；端口号建议保持默认值 8888。

串口：COM4 | USB-SERIAL CH340K (COM4)刷新串口号断开串口号芯片：auto

固件文件：C:/xuegeRos/leap_1/5.固件/leap_1_v1.1.bin选择固件...偏移地址：0x0000

☐ 烧录前整片擦除 ☐ 禁用 stub仅擦除烧录固件停止

0%

读取配置键：键名，例如 wifi_ssid值：值，例如 XueBot_AP写入配置

配置项	值
wifi_ap_pswd	
microros_mode	udp_client
udpsvr_ip	192.168.3.105
udpsvr_port	8888
ros2_nodename	xuege_motion_control
ros2_namespace	
ros2_domain_id	0

\$odom_c_frameid=base_footprint
\$twist_topic=cmd_vel
\$odom_pub_period=50
\$pid_kp=0.62
\$pid_ki=0.13
\$pid_kd=0.00[150888][V][WiFiGeneric.cpp:363] _arduino_event_cb(): STA Disconnected: SSID: xuegeros, BSSID: 00:00:00:00:00:00, Reason: 201
[150899][D][WiFiGeneric.cpp:1040] _eventCallback(): Arduino Event: 5 - STA_DISCONNECTED
[150908][W][WiFiGeneric.cpp:1062] _eventCallback(): Reason: 201 - NO_AP_FOUND
[150915][D][WiFiGeneric.cpp:1086] _eventCallback(): WiFi AutoReconnect Running
[150924][V][WiFiGeneric.cpp:97] set_esp_interface_ip(): Configuring Station static IP: 0.0.0.0, MASK: 0.0.0.0, GW: 0.0.0.0
WIFI EVENT!
[150942]@202>D:WIFI_DISCONNECTED REASON:201

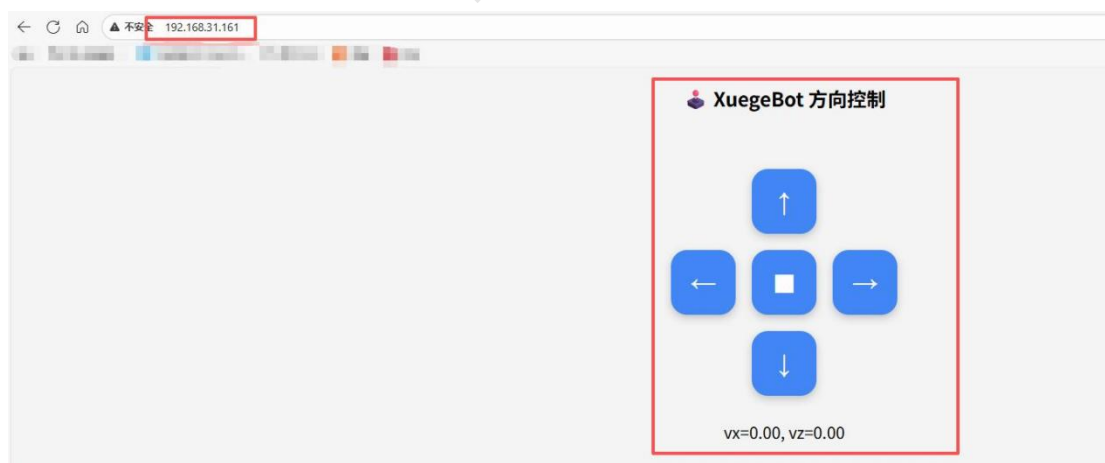
3. 保存并应用参数

所有参数填写完毕后，点击“保存参数”或“写入设备”按钮。软件将自动将配置保存到设备内部 Flash 中。建议在配置完成后重新上电一次，以确保新参数生效。



2.1.3 验证连接

参数配置完成后，设备会自动尝试连接指定的 Wi-Fi 网络。显示屏出现 IP 地址信息时，表示网络连接成功。此时在浏览器中输入屏幕显示的ip，可对机器人进行遥控操作



点击方向箭头后机器人开始运动，说明配置和连接成功

2.2 ROS 功能包编译办法

2.1 新建工作空间

打开终端新建一个工作空间

```
mkdir xuegeros_ws
```

```
cd xuegeros_ws
```

```
mkdir src
```

```
cd src
```

安装 YDLIDAR 的驱动

```
git clone https://github.com/YDLIDAR/YDLidar-SDK.git
```

```
cd YDLidar-SDK/build
```

```
cmake ..
```

```
make
```

```
sudo make install
```

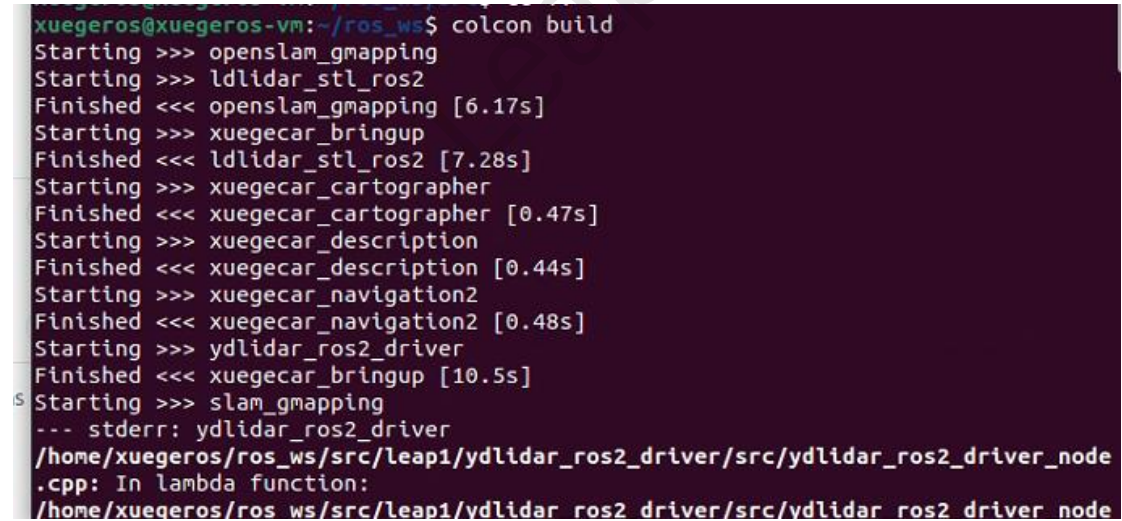
导入 Leap1 功能包

```
git clone https://gitee.com/czulcl/leap1.git
```

```
cd ../..
```

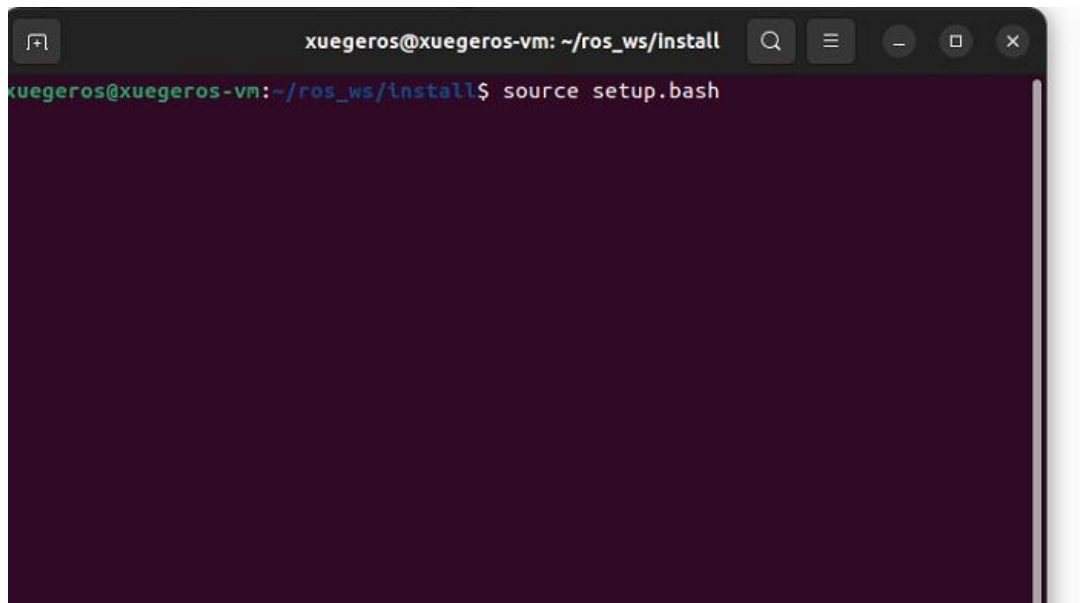
编译功能包

```
colcon build
```



```
xuegeros@xuegeros-vm:~/ros_ws$ colcon build
Starting >>> openslam_gmapping
Starting >>> ldliar_stl_ros2
Finished <<< openslam_gmapping [6.17s]
Starting >>> xuegecar_bringup
Finished <<< ldliar_stl_ros2 [7.28s]
Starting >>> xuegecar_cartographer
Finished <<< xuegecar_cartographer [0.47s]
Starting >>> xuegecar_description
Finished <<< xuegecar_description [0.44s]
Starting >>> xuegecar_navigation2
Finished <<< xuegecar_navigation2 [0.48s]
Starting >>> ydlidar_ros2_driver
Finished <<< xuegecar_bringup [10.5s]
Starting >>> slam_gmapping
--- stderr: ydlidar_ros2_driver
/home/xuegeros/ros_ws/src/leap1/ydlidar_ros2_driver/src/ydlidar_ros2_driver_node
.cpp: In lambda function:
/home/xuegeros/ros_ws/src/leap1/ydlidar_ros2_driver/src/ydlidar_ros2_driver_node
```

编译成功后 cd 到 install 目录



```
xuegeros@xuegeros-vm: ~/ros_ws/install
xuegeros@xuegeros-vm:~/ros_ws/install$ source setup.bash
```

2.2 永久配置环境变量办法

终端输入 `gedit ~/.bashrc`

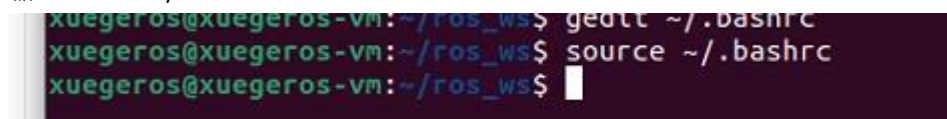
```
93 alias ls='ls -lr'
94
95 # Add an "alert" alias for long running commands.  Use like so:
96 #   sleep 10; alert
97 alias alert='notify-send --urgency=low -i "${ $? = 0 }" && echo terminal || echo error'
98
99 # Alias definitions.
100 # You may want to put all your additions into a separate file like
101 # ~/.bash_aliases, instead of adding them here directly.
102 # See /usr/share/doc/bash-doc/examples in the bash-doc package.
103
104 if [ -f ~/.bash_aliases ]; then
105     . ~/.bash_aliases
106 fi
107
108 # enable programmable completion features (you don't need to enable
109 # this, if it's already enabled in /etc/bash.bashrc and /etc/profile
110 # sources /etc/bash.bashrc).
111 if ! shopt -oq posix; then
112     if [ -f /usr/share/bash-completion/bash_completion ]; then
113         . /usr/share/bash-completion/bash_completion
114     elif [ -f /etc/bash_completion ]; then
115         . /etc/bash_completion
116     fi
117 fi
118 source /opt/ros/humble/setup.bash
119 source /home/xuegeros/xuegeros_ws/install/setup.bash
```

最下方添加 `source /xxx/xxx/xxx/install/setup.bash`

xxx 为你系统的路径

保存后退出

输入 `source ~/.bashrc`



```
xuegeros@xuegeros-vm: ~/ros_ws$ gedit ~/.bashrc
xuegeros@xuegeros-vm: ~/ros_ws$ source ~/.bashrc
xuegeros@xuegeros-vm: ~/ros_ws$
```

三、快速上手指南

3.1 查询虚拟机的 IP

虚拟机要设置为桥接模式，新建终端输入 ifconfig

```
ether fa:df:40:1a:00:4e txqueuelen 0 (Ethernet)
RX packets 0 bytes 0 (0.0 B)
RX errors 0 dropped 0 overruns 0 frame 0
TX packets 0 bytes 0 (0.0 B)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

ens33: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.31.157 netmask 255.255.255.0 broadcast 192.168.31.255
    inet6 fe80::6f5f:9e61:5072:75dd prefixlen 64 scopeid 0x20<link>
    inet6 2409:8a21:4251:ff70::f8e prefixlen 128 scopeid 0x0<global>
    inet6 2409:8a21:4251:ff70:5e7:ab8:3a7e:8bdf prefixlen 64 scopeid 0x0<global>
    inet6 2409:8a21:4251:ff70:b1c9:6c12:c0b9:c30c prefixlen 64 scopeid 0x0<global>
    ether 00:0c:29:a0:5f:1e txqueuelen 1000 (Ethernet)
    RX packets 2580 bytes 1000555 (1.0 MB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 2340 bytes 1372623 (1.3 MB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
```

找到图中 ens33 的位置，inet 显示的 ip 即为虚拟机的无线 ip

3.2 连接 leap1 的配置工具

串口: COM4 | USB-SERIAL CH340K (COM4) 刷新串口 断开串口 芯片: auto

固件文件: C:/xuegeRos/leap_1/5.固件/leap_1_v1.1.bin 选择固件... 偏移地址: 0x0000

☐ 烧录前整片擦除 ☐ 禁用 stub 仅擦除 烧录固件 停止 0%

读取配置

键: 键名, 例如 wifi_ssid 值: 值, 例如 XueBot_AP 写入配置

配置项	值
first_startup	0
serial_baud	921600
serial_id	0
wifi_ssid	xuegeros
wifi_pswd	88888888
wifi_ap_ssid	XUEBOT_82B4
wifi_ap_pswd	

```
$odom_c_frameid=base_footprint
$twist_topic=cmd_vel
$odom_pub_period=50
$pid_kp=0.62
$pid_ki=0.13
$pid_kd=0.00[150888][V][WiFiGeneric.cpp:363] _arduino_event_cb(): STA Disconnected: SSID: xuegeros, BSSID: 00:00:00:00:00:00, Reason: 201
[150899][D][WiFiGeneric.cpp:1040] _eventCallback(): Arduino Event: 5 - STA_DISCONNECTED
[150908][W][WiFiGeneric.cpp:1062] _eventCallback(): Reason: 201 - NO_AP_FOUND
[150915][D][WiFiGeneric.cpp:1086] _eventCallback(): WiFi AutoReconnect Running
[150924][V][WiFiGeneric.cpp:97] set_esp_interface_ip0: Configuring Station static IP: 0.0.0.0, MASK: 0.0.0.0, GW: 0.0.0.0
WIFI EVENT!
[150942]@202> D:WIFI DISCONNECTED REASON:201
```



配置如上图所示的参数，其中 wifi 需要连接 2.4G 频段的 wifi
udpserver_ip 输入刚刚在虚拟机里获取的 ip，udpserver_port 默认为 8888 不变。

3.3 连接 microros 代理

新建终端输入如下指令进行连接

```
sudo docker run -it --rm -v /dev:/dev -v /dev/shm:/dev/shm --privileged --net=host  
microros/micro-ros-agent:$ROS_DISTRO udp4 --port 8888 -v6
```

该步骤需要提前安装好 docker 环境并处理好网络环境，若环境不佳可选择国内镜像拉取 docker

国内指令：

该指令用于一键安装 docker

```
curl -fsSL https://get.docker.com | bash -s docker --mirror Aliyun
```

查看虚拟机版本，新建终端输入 `sudo docker version`

```
Client: Docker Engine - Community
Version:      24.0.7
API version:  1.43
Go version:   go1.20.10
Git commit:   afdd53b
Built:        Thu Oct 26 09:07:41 2023
OS/Arch:      linux/amd64
Context:      default

Server: Docker Engine - Community
Engine:
```

sudo docker run -it --rm -v /dev:/dev -v /dev/shm:/dev/shm --privileged --net=host registry.cn-hangzhou.aliyuncs.com/fishros/micro-ros-agent:humble udp4 --port 8888 -v6

该指令为国内拉取 microros_agent 指令

拉取成功后如下

```
** <<DDS>> **] | client_key: 0x00000001, len: 312, data:
0000: DD 86 12 69 80 3E 64 19 04 00 00 00 69 6D 75 00 00 00 00 40 9C 0D 7B 3F 00
    00 00 60 24 A2 79 BF
0020: 00 00 00 00 40 9D 1F C5 3F 00 00 00 E0 55 8F EF 3F 00 00 00 00 00 00 00 00
    00 00 00 00 00 00 00
0040: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
    00 00 00 00 00 00 00
0060: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
    00 00 00 93 7B B9 BF
0080: 00 00 00 80 CC 0F CB 3F 00 00 00 00 EB C2 A4 BF 00 00 00 00 00 00 00 00 00
    00 00 00 00 00 00
00A0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
    00 00 00 00 00 00 00
00C0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
    00 00 40 0E 2D 94 3F
00E0: 00 00 00 C0 1C 02 81 3F 00 00 00 A0 7C 08 F0 3F 00 00 00 00 00 00 00 00 00
    00 00 00 00 00 00 00
0100: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
    00 00 00 00 00 00 00
0120: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
[1762821853.447975] debug | UDPv4AgentLinux.cpp | send_message |
** <<UDP>> **] | client_key: 0x0F8C39FD, len: 13, data:
0000: 81 00 00 00 0A 01 05 00 21 02 00 00 80
```

3.4 启动激光雷达驱动

先输入如下指令桥接雷达数据到虚拟机

sudo socat -d -d PTY,link=/dev/lidar,raw,echo=0,mode=666 UDP4-LISTEN:8889,reuseaddr,fork

```
xuegeros@xuegeros-virtual-machine:~$ sudo socat -d -d PTY,link=/dev/lidar,raw,echo=0,mode=666 UDP4-LISTEN:8889,reuseaddr,fork
[sudo] password for xuegeros:
2025/11/13 15:37:17 socat[27598] N PTY is /dev/pts/5
2025/11/13 15:37:17 socat[27598] N listening on UDP AF=2 0.0.0.0:8889
2025/11/13 15:37:17 socat[27598] N accepting UDP connection from AF=2 192.168.31.207:62461
2025/11/13 15:37:17 socat[27598] N forked off child process 27599
2025/11/13 15:37:17 socat[27598] N listening on UDP AF=2 0.0.0.0:8889
2025/11/13 15:37:17 socat[27599] N starting data transfer loop with FDs [5,5] and [7,7]
```

再输入启动雷达驱动

ros2 launch ydlidar_ros2_driver ydlidar_launch.py


```

time 0 ms
[ydlidar_ros2_driver_node-1] [2025-11-13 15:39:59][info] Lidar init success, Elapsed time [0]ms
[ydlidar_ros2_driver_node-1] [2025-11-13 15:40:00][info] Start to getting intensity flag
[ydlidar_ros2_driver_node-1] [2025-11-13 15:40:00][info] Auto set intensity 0
[ydlidar_ros2_driver_node-1] [2025-11-13 15:40:00][info] [YDLIDAR] End to getting intensity flag
[ydlidar_ros2_driver_node-1] [2025-11-13 15:40:00][info] [YDLIDAR] Create thread 0x4E3F6640
[ydlidar_ros2_driver_node-1] [2025-11-13 15:40:00][info] Succeeded to start scan mode, Elapsed time 1062 ms
[ydlidar_ros2_driver_node-1] [2025-11-13 15:40:00][info] Scan Frequency: 10.00Hz
[ydlidar_ros2_driver_node-1] [2025-11-13 15:40:00][info] Fixed Size: 720
[ydlidar_ros2_driver_node-1] [2025-11-13 15:40:00][info] Sample Rate: 3.00K
[ydlidar_ros2_driver_node-1] [2025-11-13 15:40:00][info] Scan Frequency: 10.00Hz
[ydlidar_ros2_driver_node-1] [2025-11-13 15:40:00][info] Fixed Size: 720
[ydlidar_ros2_driver_node-1] [2025-11-13 15:40:00][info] Sample Rate: 3.00K
[ydlidar_ros2_driver_node-1] [2025-11-13 15:40:01][info] Scan Frequency: 10.00Hz
[ydlidar_ros2_driver_node-1] [2025-11-13 15:40:01][info] Fixed Size: 720
[ydlidar_ros2_driver_node-1] [2025-11-13 15:40:01][info] Sample Rate: 3.00K
[ydlidar_ros2_driver_node-1] [2025-11-13 15:40:01][info] Single Fixed Size: 240
[ydlidar_ros2_driver_node-1] [2025-11-13 15:40:01][info] Sample Rate: 3.00K
[ydlidar_ros2_driver_node-1] [2025-11-13 15:40:01][info] Succeeded to check the

```

3.5 启动 `odo->base_footprint` 的坐标变换

ros2 launch xuegecar_bringup xuegecar_bringup.launch.py

```

xuegeros@xuegeros-virtual-machine:~$ ros2 launch xuegecar_bringup xuegecar_bringup.launch.py
[INFO] [launch]: All log files can be found below /home/xuegeros/.ros/log/2025-11-13-15-44-52-255052-xuegeros-virtual-machine-27934
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [joint_state_publisher-1]: process started with pid [27935]
[INFO] [robot_state_publisher-2]: process started with pid [27937]
[INFO] [xuegecar_bringup-3]: process started with pid [27939]
[robot_state_publisher-2] [WARN] [1763019892.322414253] [robot_state_publisher]: No robot_description parameter, but command-line argument available. Assuming argument is name of URDF file. This backwards compatibility fallback will be removed in the future.
[robot_state_publisher-2] [INFO] [1763019892.324955868] [robot_state_publisher]: got segment base_footprint
[robot_state_publisher-2] [INFO] [1763019892.324986118] [robot_state_publisher]: got segment base_link
[robot_state_publisher-2] [INFO] [1763019892.324989978] [robot_state_publisher]: got segment caster_link
[robot_state_publisher-2] [INFO] [1763019892.324992614] [robot_state_publisher]: got segment imu_link
[robot_state_publisher-2] [INFO] [1763019892.324995039] [robot_state_publisher]: got segment laser_frame
[robot_state_publisher-2] [INFO] [1763019892.324997276] [robot_state_publisher]: got segment left_wheel_link

```

3.6 开启 `gmapping` 建图:

ros2 launch xuegecar_navigation2 slam_nav2.launch.py

```

xuegeros@xuegeros-virtual-machine:~$ ros2 launch xuegecar_bringup xuegecar_bringup.launch.py
[INFO] [launch]: All log files can be found below /home/xuegeros/.ros/log/2025-11-13-15-44-52-255052-xuegeros-virtual-machine-27934
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [joint_state_publisher-1]: process started with pid [27935]
[INFO] [robot_state_publisher-2]: process started with pid [27937]
[INFO] [xuegecar_bringup-3]: process started with pid [27939]
[robot_state_publisher-2] [WARN] [1763019892.322414253] [robot_state_publisher]: No robot_description parameter, but command-line argument available. Assuming argument is name of URDF file. This backwards compatibility fallback will be removed in the future.
[robot_state_publisher-2] [INFO] [1763019892.324955868] [robot_state_publisher]: got segment base_footprint
[robot_state_publisher-2] [INFO] [1763019892.324986118] [robot_state_publisher]: got segment base_link
[robot_state_publisher-2] [INFO] [1763019892.324989978] [robot_state_publisher]: got segment caster_link
[robot_state_publisher-2] [INFO] [1763019892.324992614] [robot_state_publisher]: got segment imu_link
[robot_state_publisher-2] [INFO] [1763019892.324995039] [robot_state_publisher]: got segment laser_frame
[robot_state_publisher-2] [INFO] [1763019892.324997276] [robot_state_publisher]: got segment left_wheel_link

```

3.7 开启 nav2 导航

ros2 launch xuegecar_navigation2 slam_nav2.launch.py

启动该指令后会自动打开 rviz2 并加载相关指令

四、Leap1 组件测试

4.1 Microros 通讯测试

4.1.1 连接 Micro-ROS 代理

1. 启动代理容器

在虚拟机中打开终端，输入以下命令启动 Micro-ROS Agent 容器：

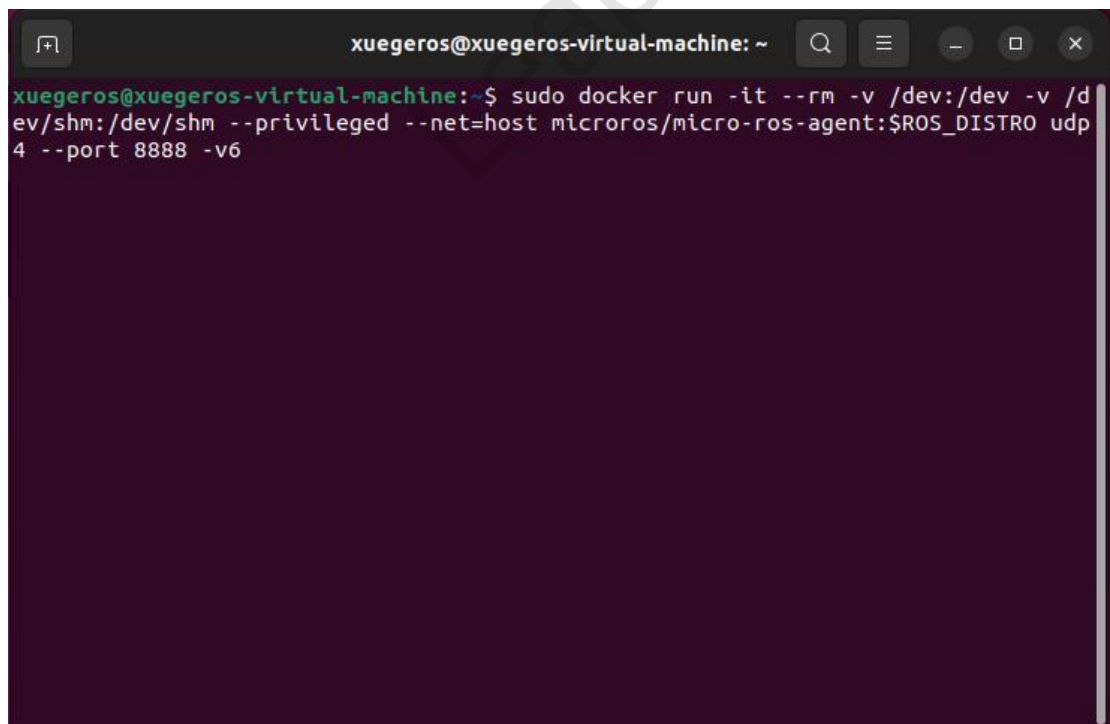
```
sudo docker run -it --rm \
-v /dev:/dev \
-v /dev/shm:/dev/shm \
--privileged \
--net=host \
microros/micro-ros-agent:humble udp4 --port 8888-v4
```

说明：

--privileged：允许容器访问硬件设备；

--net=host：使用主机网络模式，使机器人能直接访问；

--port 8888：设置 UDP 通讯端口；

A terminal window titled 'xuegeros@xuegeros-virtual-machine: ~' with search, menu, and window control icons. The terminal shows the command: 'xuegeros@xuegeros-virtual-machine:~\$ sudo docker run -it --rm -v /dev:/dev -v /dev/shm:/dev/shm --privileged --net=host microros/micro-ros-agent:\$ROS_DISTRO udp4 --port 8888 -v6'. The command is split across multiple lines. The terminal background is dark purple, and the text is white and green. A vertical scrollbar is visible on the right side of the terminal window.

```
xuegeros@xuegeros-virtual-machine:~$ sudo docker run -it --rm -v /dev:/dev -v /dev/shm:/dev/shm --privileged --net=host microros/micro-ros-agent:$ROS_DISTRO udp4 --port 8888 -v6
```

2. 连接机器人

打开机器人电源。

机器人将自动尝试连接至上一步启动的 Micro-ROS Agent。

若连接成功，终端会出现打印 microros 的相关数据。


```

** <<DDS>> **] | client_key: 0x00000001, len: 312, data:
0000: DD 86 12 69 80 3E 64 19 04 00 00 00 69 6D 75 00 00 00 00 40 9C 0D 7B 3F 00
00 00 60 24 A2 79 BF
0020: 00 00 00 40 9D 1F C5 3F 00 00 00 E0 55 8F EF 3F 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00
0040: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00
0060: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 93 7B B9 BF
0080: 00 00 00 80 CC 0F CB 3F 00 00 00 00 EB C2 A4 BF 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00
00A0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00
00C0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 40 0E 2D 94 3F
00E0: 00 00 00 C0 1C 02 81 3F 00 00 00 A0 7C 08 F0 3F 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00
0100: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00
0120: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
[1762821853.447975] debug | UDPv4AgentLinux.cpp | send_message |
[** <<UDP>> **] | client_key: 0x0F8C39FD, len: 13, data:
0000: 81 00 00 00 0A 01 05 00 21 02 00 00 80

```

3. 连接失败时的检查

若连接未成功，请依次检查以下内容：

机器人屏幕上显示的 IP 地址 是否与虚拟机的 IP 地址相同；

虚拟机的防火墙是否关闭或放行了 UDP 8888 端口；

网络是否在同一网段（例如 192.168.x.x）；

确认容器正常运行并未退出。

4.1.2 测试 ROS2 话题

1. 查看话题列表

在 ROS 2 环境的终端中输入以下命令：

```
ros2 topic list
```

执行后会列出系统中当前所有活跃的话题

```

xuegeros@xuegeros-virtual-machine:~$ ros2 topic list
/battery_state
/cmd_vel
/imu
/odom
/parameter_events
/rosout
xuegeros@xuegeros-virtual-machine:~$

```

2. 订阅单个话题

例如，测试 IMU 数据，可输入：

```
ros2 topic echo /imu
```


如果通讯正常，终端会连续输出如下数据：

```
---
header:
  stamp:
    sec: 1763019048
    nanosec: 616000000
  frame_id: imu
orientation:
  x: -0.010611345060169697
  y: 0.005900831427425146
  z: 0.9160065054893494
  w: 0.400979608297348
orientation_covariance: '<array type: double[9]>'
angular_velocity:
  x: 0.17587816715240479
  y: -0.07917559146881104
  z: 0.1239081621170044
angular_velocity_covariance: '<array type: double[9]>'
linear_acceleration:
  x: -0.02387426793575287
  y: -0.0065673827193677425
  z: 1.0177699327468872
linear_acceleration_covariance: '<array type: double[9]>'
---
```

4.2 Gmapping 建图

本章节将介绍如何在 ROS 2 环境下完成 激光雷达数据接入、代理连接、数据可视化以及 SLAM 建图流程。

4.2.1 启动 micro-ROS 代理并连接机器人

在使用激光雷达建图之前，需要保证机器人能够通过 micro-ROS 与虚拟机正常通信。以提供的 Ubuntu 22.04 虚拟机为例，可在终端中输入以下命令启动 micro-ROS Agent：

```
sudo docker run -it --rm \
-v /dev:/dev \
-v /dev/shm:/dev/shm \
--privileged --net=host \
microros/micro-ros-agent:humble udp4 \
--port 8888-v6
```

执行该指令后，micro-ROS Agent 会在本机开启 UDP 端口 **8888** 并等待机器人上线。

打开机器人电源开关，系统会自动尝试连接代理。若连接成功，终端将持续显示来自设备的心跳信息与话题注册信息。此时 micro-ROS 通讯链路已建立，可继续执行后续步骤。

4.2.2 启动激光雷达（YDLIDAR X2）

由于 Leap1 采用虚拟机作为 ROS2 主控，需要通过虚拟串口的方式将来自机器人侧的 UDP 雷达数据重新映射到虚拟机本地的 `/dev/lidar` 设备。

2.1 使用 socat 创建虚拟串口

在另一个终端执行：

```
sudo socat -d -d \
PTY,link=/dev/lidar,raw,echo=0,mode=666 \
UDP4-LISTEN:8889,reuseaddr,fork
```

```
xuegeros@xuegeros-virtual-machine:~$ sudo socat -d -d PTY,link=/dev/lidar,raw,echo=0,mode=666 UDP4-LISTEN:8889,reuseaddr,fork
[sudo] password for xuegeros:
2025/11/13 15:37:17 socat[27598] N PTY is /dev/pts/5
2025/11/13 15:37:17 socat[27598] N listening on UDP AF=2 0.0.0.0:8889
2025/11/13 15:37:17 socat[27598] N accepting UDP connection from AF=2 192.168.31.207:62461
2025/11/13 15:37:17 socat[27598] N forked off child process 27599
2025/11/13 15:37:17 socat[27598] N listening on UDP AF=2 0.0.0.0:8889
2025/11/13 15:37:17 socat[27599] N starting data transfer loop with FDs [5,5] and [7,7]
```

上述命令将：

监听来自机器人 UDP 端口 8889 的激光雷达数据；

创建名为 /dev/lidar 的虚拟串口设备；

将 UDP 数据实时写入虚拟串口供 ydlidar_ros2_driver 使用。

执行以下命令确认虚拟串口是否成功创建：

ls /dev/lidar

终端显示 /dev/lidar 即表明映射建立成功。

```
xuegeros@xuegeros-virtual-machine:~$ ls /dev/lidar
/dev/lidar
xuegeros@xuegeros-virtual-machine:~$
```

4.2.3 启动 YDLIDAR ROS2 驱动

继续打开一个新终端，运行激光雷达驱动功能包：

ros2 launch ydlidar_ros2_driver ydlidar_launch.py

若驱动启动正常，终端将输出一系列雷达初始化信息，包含：

雷达型号识别

转速检测

串口连接状态

点云与扫描频率

此时激光雷达已成功进入工作状态。

```

time 0 ms
[ydlidar_ros2_driver_node-1] [2025-11-13 15:39:59][info] Lidar init success, Elapsed time [0]ms
[ydlidar_ros2_driver_node-1] [2025-11-13 15:40:00][info] Start to getting intensity flag
[ydlidar_ros2_driver_node-1] [2025-11-13 15:40:00][info] Auto set intensity 0
[ydlidar_ros2_driver_node-1] [2025-11-13 15:40:00][info] [YDLIDAR] End to getting intensity flag
[ydlidar_ros2_driver_node-1] [2025-11-13 15:40:00][info] [YDLIDAR] Create thread 0x4E3F6640
[ydlidar_ros2_driver_node-1] [2025-11-13 15:40:00][info] Succeeded to start scan mode, Elapsed time 1062 ms
[ydlidar_ros2_driver_node-1] [2025-11-13 15:40:00][info] Scan Frequency: 10.00Hz
[ydlidar_ros2_driver_node-1] [2025-11-13 15:40:00][info] Fixed Size: 720
[ydlidar_ros2_driver_node-1] [2025-11-13 15:40:00][info] Sample Rate: 3.00K
[ydlidar_ros2_driver_node-1] [2025-11-13 15:40:00][info] Scan Frequency: 10.00Hz
[ydlidar_ros2_driver_node-1] [2025-11-13 15:40:00][info] Fixed Size: 720
[ydlidar_ros2_driver_node-1] [2025-11-13 15:40:00][info] Sample Rate: 3.00K
[ydlidar_ros2_driver_node-1] [2025-11-13 15:40:01][info] Scan Frequency: 10.00Hz
[ydlidar_ros2_driver_node-1] [2025-11-13 15:40:01][info] Fixed Size: 720
[ydlidar_ros2_driver_node-1] [2025-11-13 15:40:01][info] Sample Rate: 3.00K
[ydlidar_ros2_driver_node-1] [2025-11-13 15:40:01][info] Single Fixed Size: 240
[ydlidar_ros2_driver_node-1] [2025-11-13 15:40:01][info] Sample Rate: 3.00K
[ydlidar_ros2_driver_node-1] [2025-11-13 15:40:01][info] Succeeded to check the

```

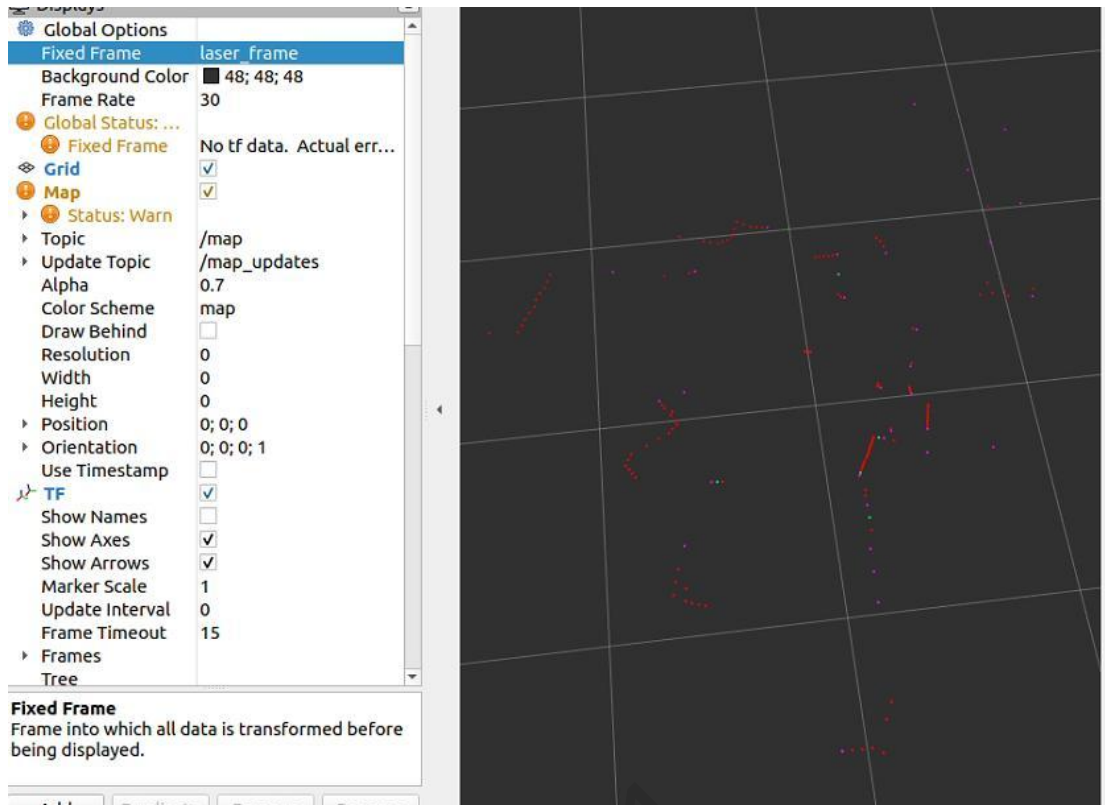
4.2.4 使用 RViz2 可视化雷达数据

在另一个终端中输入：

```
rviz2
```

进入 RViz2 后，请将 **Fixed Frame** 设置为：laser_frame

随后添加 LaserScan 显示插件，即可实时观察雷达扫描效果。若扫描图正常、无明显噪声，则说明前期的数据桥接与驱动加载均正常工作。



4.2.5 启动里程计与底盘 TF

在进行 SLAM 建图前，需先由机器人上报 `odom` $\rightarrow$ `base_footprint` 的 TF 变换数据。

执行：

`ros2 launch xuegecar_bringup xuegecar_bringup.launch.py`

```
xuegeros@xuegeros-virtual-machine:~$ ros2 launch xuegecar_bringup xuegecar_bringup.launch.py
[INFO] [launch]: All log files can be found below /home/xuegeros/.ros/log/2025-11-13-15-44-52-255052-xuegeros-virtual-machine-27934
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [joint_state_publisher-1]: process started with pid [27935]
[INFO] [robot_state_publisher-2]: process started with pid [27937]
[INFO] [xuegecar_bringup-3]: process started with pid [27939]
[robot_state_publisher-2] [WARN] [1763019892.322414253] [robot_state_publisher]: No robot_description parameter, but command-line argument available. Assuming argument is name of URDF file. This backwards compatibility fallback will be removed in the future.
[robot_state_publisher-2] [INFO] [1763019892.324955868] [robot_state_publisher]: got segment base_footprint
[robot_state_publisher-2] [INFO] [1763019892.324986118] [robot_state_publisher]: got segment base_link
[robot_state_publisher-2] [INFO] [1763019892.324989978] [robot_state_publisher]: got segment caster_link
[robot_state_publisher-2] [INFO] [1763019892.324992614] [robot_state_publisher]: got segment imu_link
[robot_state_publisher-2] [INFO] [1763019892.324995039] [robot_state_publisher]: got segment laser_frame
[robot_state_publisher-2] [INFO] [1763019892.324997276] [robot_state_publisher]: got segment left_wheel_link
```

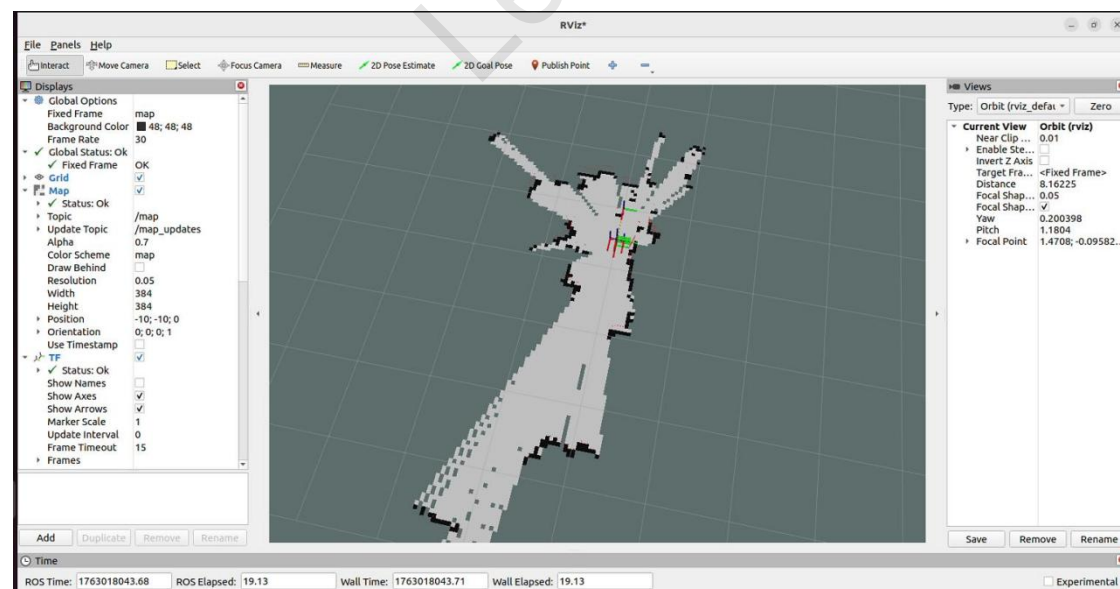

4.2.6 运行 Gmapping 进行激光建图

新建终端，启动 gmapping 功能包：

ros2 launch slam_gmapping slam_gmapping.launch.py

```
xuegeros@xuegeros-virtual-machine:~$ ros2 launch slam_gmapping slam_gmapping.lau
nch.py
[INFO] [launch]: All log files can be found below /home/xuegeros/.ros/log/2025-1
1-13-15-13-25-830590-xuegeros-virtual-machine-25601
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [slam_gmapping-1]: process started with pid [25602]
[slam_gmapping-1] [INFO] [1763018007.694026087] [slam_gmapping]: Message Filter
dropping message: frame 'laser_frame' at time 1763018006.815 for reason 'discar
ding message because the queue is full'
[slam_gmapping-1] [INFO] [1763018007.781096003] [slam_gmapping]: Message Filter
dropping message: frame 'laser_frame' at time 1763018006.903 for reason 'discar
ding message because the queue is full'
[slam_gmapping-1] [INFO] [1763018008.063891541] [slam_gmapping]: Message Filter
dropping message: frame 'laser_frame' at time 1763018007.136 for reason 'discar
ding message because the queue is full'
[slam_gmapping-1] [INFO] [1763018008.081790943] [slam_gmapping]: Message Filter
dropping message: frame 'laser_frame' at time 1763018007.145 for reason 'discar
ding message because the queue is full'
[slam_gmapping-1] [INFO] [1763018008.081903628] [slam_gmapping]: Laser is mounte
d upwards.
[slam_gmapping-1] [INFO] [1763018008.082081754] [slam_gmapping]: Initialization
complete
[slam_gmapping-1] Laser Pose= 0.565674 0.00327823 -0.00225426
[slam_gmapping-1] -maxUrange 11.99 -maxUrange 11.99 -sigma 0.05 -kernelSize
```

GMapping 节点将订阅 /scan 与 /odom 两类数据，并实时生成二维栅格地图。若一切正常，RViz2 中“Map”模块会不断更新环境的占据栅格。



为了构建高质量地图，需要使机器人围绕场地缓慢移动。

在新终端发布速度控制指令：

新建一个终端用来发布速度命令，控制 leap_1 进行移动建图


```
xuegeros@xuegeros-virtual-machine: ~ $ ros2 launch xuegeros_b xuegeros_b.launch.py
b : down (-z)

anything else : stop

q/z : increase/decrease max speeds by 10%
w/x : increase/decrease only linear speed by 10%
e/c : increase/decrease only angular speed by 10%

CTRL-C to quit

currently:      speed 0.50      turn 1.00
currently:      speed 0.45      turn 1.00
currently:      speed 0.41      turn 1.00
currently:      speed 0.36      turn 1.00
currently:      speed 0.33      turn 1.00
currently:      speed 0.30      turn 1.00
currently:      speed 0.27      turn 1.00
currently:      speed 0.24      turn 1.00
currently:      speed 0.22      turn 1.00
currently:      speed 0.19      turn 1.00
currently:      speed 0.19      turn 0.90
currently:      speed 0.19      turn 0.81
currently:      speed 0.19      turn 0.73
```

建议将线速度和角速度设置的慢一些，建图的效果会更好
随着机器人移动，GMMapping 会不断叠加激光扫描，并实时构建环境平面图。

地图话题为 map

ROS2 的地图服务为 nav2_map_server 的工具

我们的地图话题为 map，文件名字我们用 room,所以有下面这个这样写的命令行。

```
ros2 run nav2_map_server map_saver_cli -t map -f room
```

4.2.7 运行 Cartography 进行激光建图

注意：本示例采用的是纯激光雷达建图，也就是只需要 /scan 数据的输入
新建终端，启动 cartography 功能包：

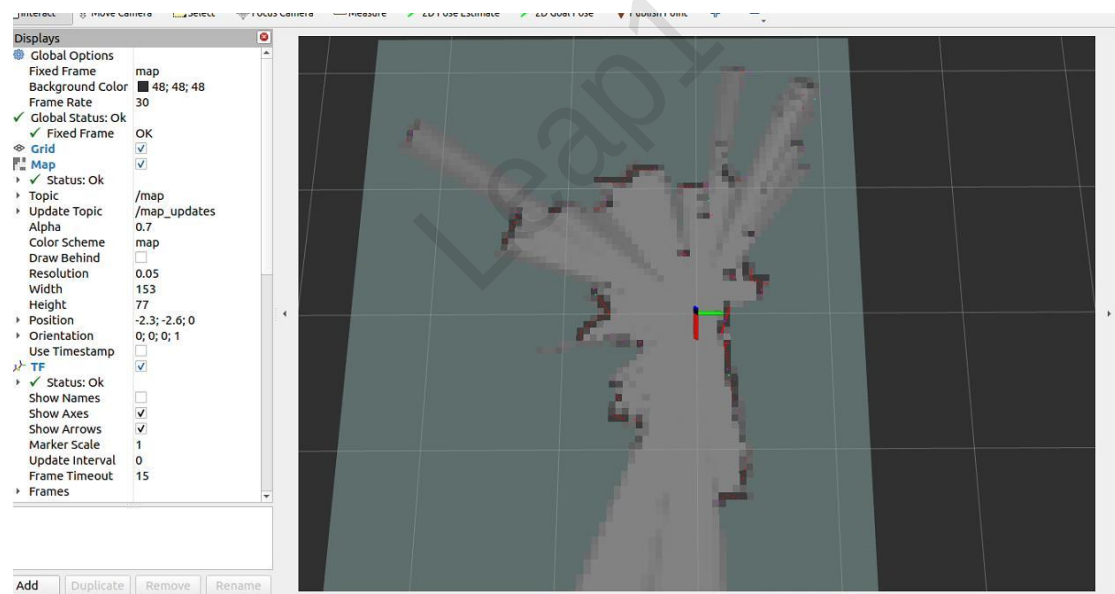
```
ros2 launch xuegecar_cartography cartography.launch.py
```

```

xuegeros@xuegeros-virtual-machine: ~ 80x24
xuegeros@xuegeros-virtual-machine:~$ ros2 launch xuegecar_cartographer cartograp
her.launch.py
[INFO] [launch]: All log files can be found below /home/xuegeros/.ros/log/2025-1
1-13-15-10-13-939614-xuegeros-virtual-machine-25075
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [cartographer_node-1]: process started with pid [25076]
[INFO] [cartographer_occupancy_grid_node-2]: process started with pid [25078]
[INFO] [rviz2-3]: process started with pid [25080]
[rviz2-3] Warning: Ignoring XDG_SESSION_TYPE=wayland on Gnome. Use QT_QPA_PLATFO
RM=wayland to run on Wayland anyway.
[cartographer_node-1] [INFO] [1763017814.010827764] [cartographer logger]: I1113
15:10:14.000000 25076 configuration_file_resolver.cc:41] Found '/home/xuegeros/
xuegeros_ws/install/xuegecar_cartographer/share/xuegecar_cartographer/config/xue
gecar_laser_2d.lua' for 'xuegecar_laser_2d.lua'.
[cartographer_node-1] [INFO] [1763017814.011059170] [cartographer logger]: I1113
15:10:14.000000 25076 configuration_file_resolver.cc:41] Found '/opt/ros/humble
/share/cartographer/configuration_files/map_builder.lua' for 'map_builder.lua'.
[cartographer_node-1] [INFO] [1763017814.011069745] [cartographer logger]: I1113
15:10:14.000000 25076 configuration_file_resolver.cc:41] Found '/opt/ros/humble
/share/cartographer/configuration_files/map_builder.lua' for 'map_builder.lua'.
[cartographer_node-1] [INFO] [1763017814.011087416] [cartographer logger]: I1113
15:10:14.000000 25076 configuration_file_resolver.cc:41] Found '/opt/ros/humble
/share/cartographer/configuration_files/pose_graph.lua' for 'pose_graph.lua'.
[cartographer_node-1] [INFO] [1763017814.011094024] [cartographer logger]: I1113

```

Cartography 节点将订阅 /scan 数据，并实时生成二维栅格地图。若一切正常，RViz2 中 “Map” 模块会不断更新环境的占据栅格。



4.3 Nav2 导航测试

在完成建图并保存地图后，即可进行 Nav2 导航功能的验证。本章节将介绍如何加载地图、启动导航系统、设定初始位姿以及执行目标导航任务，帮助用户快速完成从地图到自主导航的完整流程。

4.3.1 启动 Nav2 导航系统

确保系统已正确加载建图阶段生成的地图文件，并确认 micro-ROS 与机器人底盘均已连接。

在终端中输入：

```
ros2 launch xuegecar_navigation2 navigation2.launch.py
```

注意，如果使用保存的地图

```
ros2 launch xuegecar_navigation2 navigation2.launch.py map:=/path/to/your/map.yaml
```

/path/to/your/map.yaml 为地图的路径

执行该命令后，导航系统将依次启动如下核心组件：

map_server：加载静态地图

amcl：基于粒子滤波的自适应蒙特卡洛定位

bt_navigator：行为树路径执行器

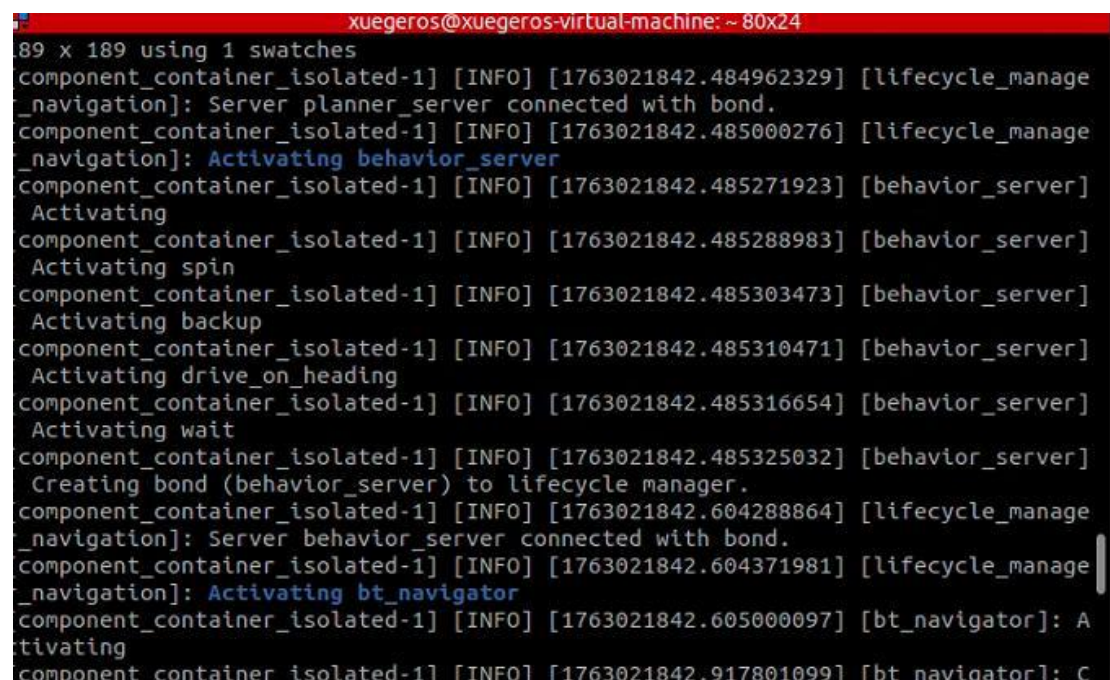
planner_server：全局路径规划

controller_server：局部路径跟踪

recoveries_server：恢复行为

lifecycle_manager：管理导航生命周期

终端会显示各节点激活状态，并在完成初始化后自动载入地图。



```
xuegeros@xuegeros-virtual-machine: ~ 80x24
89 x 189 using 1 swatches
component_container_isolated-1 [INFO] [1763021842.484962329] [lifecycle_manage_navigation]: Server planner_server connected with bond.
component_container_isolated-1 [INFO] [1763021842.485000276] [lifecycle_manage_navigation]: Activating behavior_server
component_container_isolated-1 [INFO] [1763021842.485271923] [behavior_server] Activating
component_container_isolated-1 [INFO] [1763021842.485288983] [behavior_server] Activating spin
component_container_isolated-1 [INFO] [1763021842.485303473] [behavior_server] Activating backup
component_container_isolated-1 [INFO] [1763021842.485310471] [behavior_server] Activating drive_on_heading
component_container_isolated-1 [INFO] [1763021842.485316654] [behavior_server] Activating wait
component_container_isolated-1 [INFO] [1763021842.485325032] [behavior_server] Creating bond (behavior_server) to lifecycle manager.
component_container_isolated-1 [INFO] [1763021842.604288864] [lifecycle_manage_navigation]: Server behavior_server connected with bond.
component_container_isolated-1 [INFO] [1763021842.604371981] [lifecycle_manage_navigation]: Activating bt_navigator
component_container_isolated-1 [INFO] [1763021842.605000097] [bt_navigator]: Activating
component_container_isolated-1 [INFO] [1763021842.917801099] [bt_navigator]: C
```

4.3.2 地图加载与显示

启动完成后，RViz2 会自动加载 map_server 所提供的静态地图。

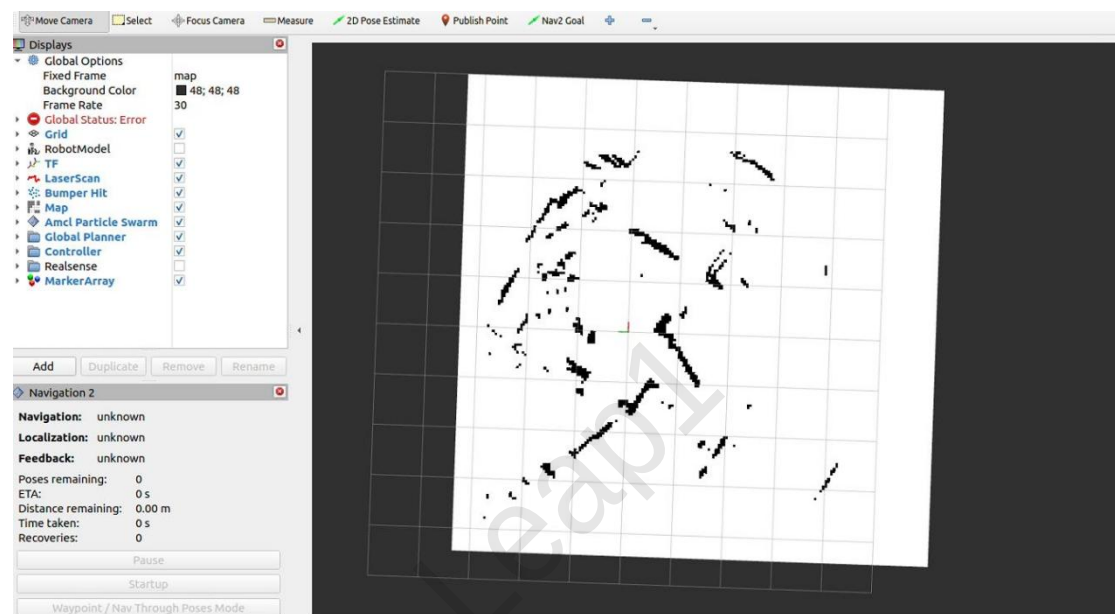
在 RViz2 左侧的“Displays”面板中应能看到：

Map（来自 /map）

LaserScan（来自 /scan）

TF（机器人坐标关系）

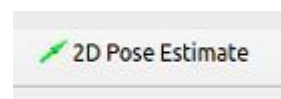
若地图正常显示，即可进行下一步定位操作



4.3.3 设置机器人初始位姿（2D Pose Estimate）

由于 AMCL 使用概率定位方法，系统需要一个合理的初始位置作为粒子云的起点。

在 RViz2 工具栏中点击：



然后在地图中单击并拖拽箭头，设定：

机器人在地图中的 XY 位置

机器人朝向（箭头方向）

设置后，AMCL 将开始接收激光数据、里程计数据与地图进行匹配。

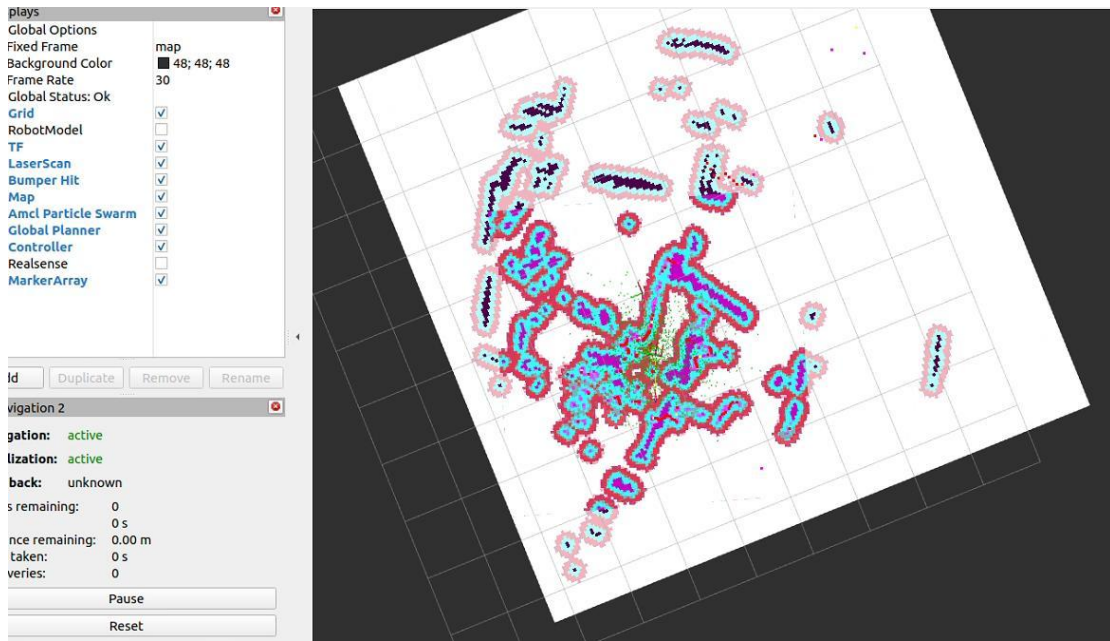
RViz2 中可观察到：

粒子云逐渐收敛

机器人姿态在地图中稳定显示

局部代价地图（Local Costmap）开始生成

此时说明定位已成功。



4.3.4 导航系统各组件的实时反馈

定位成功后，RViz2 中会自动呈现：

1. 局部代价地图（Local Costmap）

由激光雷达数据实时生成，用于动态避障。

2. 膨胀层（Inflation Layer）

依据机器人尺寸自动生成安全边界，使规划路径更平滑。

3. 当前局部路径规划器（DWB）

本项目默认使用 DWB（Dynamic Window Approach）作为局部控制器。

其职责包括：

根据障碍物情况动态优化速度指令

保证运动平滑

实现避障行为

4.3.5 执行目标导航任务

当定位稳定后，可开始执行自主导航任务。

在 RViz2 工具栏选择：

在地图上点击目标位置并拖动方向箭头以指定朝向，系统将自动执行：

全局路径规划（Navfn / Smac）

局部路径生成（DWB 控制器）

速度指令发布（/cmd_vel）

避障与路径跟踪

若机器人成功到达目标点，导航节点将在终端输出成功信息。

Leap1